ISA

Küçük Mesaj içinde √N, i√N, 1/N, ΣN ve N! ifadelerini tür olarak taşımayı anlatıyorum. Her ifadeyi hemen açmadan sayının anlamını korumak istiyorum. Türü takmakla açılmış değerini hesaplamak ayrı işler. Etiketin de saklama ve kontrol maliyeti var.

ISA, komut sözleşmesi. Programın makineden ne isteyebildiğini, bu isteğin nasıl yazıldığını ve çalışınca neyin değiştiğini anlatıyor. Aşağıda 192 bitlik Hunter tasarımını kullanıyorum. Temmuz 2026 komut haritasını kendi tarihsel bölümünde tutuyorum, çünkü iki harita aynı kodların bazılarına farklı anlam veriyor.

Sayı türleri

Tür, `rtyp` alanının 0 numaralı hücresinde. Diğer yedi hücre açık. Burada N, sayı kaydının taşıdığı büyüklük veya ifadenin girdisi.

Kod	Ad	Anlam
0	POS	+N
1	NEG	-N
2	SQRT	√N
3	ISQRT	√(-N), negatif olmayan N için i√N
4	INV	1/(+N)
5	NINV	1/(-N)
6	SIGMA	1+2+...+N
7	FACT	N!

Bu tablo tür tasarımını anlatıyor. Her türün güncel kodda bütünüyle yürütüldüğü anlamına gelmiyor. Arşivdeki Hunter çekirdeği yalnız 0 ve 1 atıyor. Tanım kümesi kontrolleri, sıfır payda ve diğer türleri açma kuralları için uygulama sözleşmesi gerekiyor. Negatif değer elde etmek için 24 bitlik büyüklüğü sessizce işaretli makine tamsayısı saymıyorum.

Hunter komut kelimesi

Bir komut kelimesi dört xbit, yani 12 bit:

```
[ OP | MOD | A | B ]
   3     3    3   3 bit
```

`OP` işlem ailesini seçiyor. `MOD` anlamını değiştiriyor. Aritmetik örneklerinde A hem hedefi hem ilk kaynak register'ı, B ikinci kaynak register'ı gösteriyor. 0..7 haneleri `R0..R7`yi adlandırabiliyor. Diğer operand ve adresleme kullanımlarının hepsi kararlaştırılmış değil.

OP	Aile	Anlamı ve bugünkü sınırı
0	LOAD	Değeri yükler, gölge kurmaz. Tam genişlikte sabitin nereden geldiği açık
1	MOV	Sayıyı gölgesiyle birlikte taşır
2	ADD	ADD, SUB ve CMP
3	MUL	SMUL, SDIV, SQRT, SQR, SHL ve SHR. S, shadow yani gölge
4	J	MOD ile koşullu veya koşulsuz atlama seçer. Hedef kodlaması açık
5	LD	LD R3,[R5] gibi bellekten okur. Kodda yok
6	ST	ST [R5],R3 gibi belleğe yazar. Kodda yok
7	SYS	Durma, sistem ve genişletme girişi. Alt kodlar açık

ADD ve MUL ailelerinde MOD bitlerinin işleri şöyle:

Bit	Görev
0	İşaret/ters, ADD ailesinde SUB, normal MUL ailesinde SDIV
1	Genişletilmiş işlem
2	CMP gibi işlemlerde normal sonuç yazımını kapatma

ADD, MOD 0 kullanıyor. SUB, MOD 1 kullanıyor. CMP, çıkarmayla normal aritmetik sonuç yazımını kapatmayı birleştirip MOD 5 kullanıyor. CMP yine karşılaştırma/durum bilgisini saklıyor. Hangi hücreye yazdığı açık. J'nin MOD'u kendi karşılaştırmasını seçiyor. Bu yüzden her atlamadan önce CMP gerektirmiyorum.

MUL için önerilen genişletilmiş havuz `000=SQRT`, `001=SQR`, `010=SHL`, `011=SHR`, `111=genişletme`. Bunun komut kodlamasındaki yeri açık. Genişletme biti seçilince MOD'da yalnız iki bit kalıyor, havuz ise üç bit istiyor. Henüz her genişletilmiş işlem için tam makine kelimesi veremiyorum.

SHL ve SHR bir sekizlik hane, yani üç bit kaydırıyor. SHL, tutulan genişlik içinde ×8'e karşılık geliyor. SHR son sekizlik haneyi düşürüyor. Sınırın dışına düşen hane, `rpow` değiştirilerek korunmuyor.

Doğrudan okuyabildiğim üç kelime

Buradaki dört haneli komut kelimeleri sekizlik.

Kelime	Alanlar	Anlam
2035_8	OP=2, MOD=0, A=3, B=5	R3 = R3 + R5
2135_8	OP=2, MOD=1, A=3, B=5	R3 = R3 - R5
2535_8	OP=2, MOD=5, A=3, B=5	Çıkararak karşılaştır, normal sonuç yazımını kapat, durumu sakla

Küçük bir pozitif örnekte `R3.rm=6`, `R5.rm=7` ile başlıyorum. İkisi de düz pozitif türde, ölçekleri sıfır. `2035_8` bu iki değeri okuyor, pozisyonel toplayıcıya gönderiyor ve R3'e 13 yazıyor. R5'te 7 kalıyor. R3'ün eski gölgesi güncellenmeli veya geçersiz sayılmalı. Bu örnekte büyüklük taşması yok. Hâlâ açık olan durum hücresi yerleşimini veya komut zamanlamasını tanımlamıyor.

J koşulları

Burada A ve B'nin seçtiği operandları a ve b diye okuyorum.

MOD	Koşul
0	a < b
1	a = b
2	a > b
3	Tanımsız
4	a, b'yi böler
5	b, a'yı böler
6	obeb(a,b)=1
7	Koşulsuz

Bu taslakta `JMP=J.7`, `JE=J.1`, `JL=J.0`, `JG=J.2` takma adlarını kullanıyorum. Pozitif, tek satırlı smooth değerlerde `rc=1` ise bölünebilirlik, karşılık gelen asalların üslerini karşılaştırarak sınanabiliyor. Alt satırlar veya rough varsa bu kısa yol yetmiyor, genel yol açık. Aralarında asallık koşulu için önerilen üs yolu da açık. Atlama hedefinin dolu olan dört alanda ayrılmış yeri yok.

Yürütme Mantığı

Komuttan önce giriş register'ları ve program hazır. Program sayacı komutu gösteriyor. Getirme işi, komut kelimesini kontrolün önüne getiriyor. Kontrol OP, MOD ve operand seçicilerini okuyor, ilgili yolu seçiyor.

```
program sayacı → komut kelimesi → operand okumaları
                                          |
                                normal veya gölge işlemi
                                          |
                                sonuç ve durum yazımları
                                          |
                          sıradaki komut veya atlama hedefi
```

ADD örneğinde mantıksal iş bir komut getirme, iki kaynak register'ı okuma, bir toplama ve bir hedef değer yazımı. Durum bilgisi ile gölgenin bakımı bunlara ek görevler. Bu sayılar fiziksel bellek erişimi veya saat çevrimi iddiası değil.

Döngü, bir atlama önceki komutu seçtiği için tekrar ediyor. Ne zaman biteceğini döngünün kendi sayacı veya koşulu belirliyor. SYS üzerinden durma hedefleniyor ama bit kalıbı açık. Hedef, durma ve operand kodlamaları tamamlanana kadar yalnız bu sayfayla çalıştırılabilir bir programın bütünüyle tanımlandığını söyleyemem. Yürütme görünümünde komut seçiminin, operand okumalarının ve yazılmış sonucun ayrı durumlar olarak görünmesini bekliyorum.

Hunter sözleşmesinin açıkları

LD ve ST arşivdeki kodda uygulanmış değil.

SYS alt kodları ve J hedef kodlaması açık.

LOAD'un dört haneli komuttan sekiz haneli sabite ulaşma yolu belirlenmedi.

Genişletilmiş MUL alt kodlarının yeri açık.

Durum, CMP çıktısı ve ölçeğin ayrıntılı hücreleri açık.

Genel rough bölünebilirlik dalları, gölgenin bitiş kodlaması ve `rc` yorumu açık.

Taşma davranışı, bölme istisnaları, adresleme biçimleri ve tam zamanlama tablosu burada tanımlı değil.

Opcode seçiminin mikrokod olmadan doğrudan kontrol yollarını sürmesini hedefliyorum. Bu hedef de seçim mantığı, saklama ve bağlantı gerektiriyor. Bir sentez sonucu veya ölçülmüş donanım maliyeti değil.

Önceki sürüm: Temmuz 2026 kilitli ISA

Bu, 9 Temmuz 2026'da kilitlenen önceki harita. v1.0.3 içeriği 12 Temmuz tarihli. Metni 1 Ekim'de yeniden çıkarılmış. Metin çıkarımının sonraki tarihi, onu Eylül Hunter haritasına dönüştürmüyor.

```
[ op | MOD | dst | src/imm ]
  her alan bir xbit, toplam dört xbit
```

OP	Temmuz anlamı
0	LOAD, tembel gölge
1	MOV, gölge değerle birlikte taşınır
2	ADD ailesi
3	MUL ailesi
4	JMP
5	JEQ
6	JNE
7	SYS kaçış kodu

Bu önceki sözleşmede büyük sabitler ek kelime kullanıyor. MOD bit 0 işaret/ters, bit 1 genişletme, bit 2 normal sonuç yazımını kapatma. Genişletilmiş havuz SQRT ve SQR'ı listeliyor, `010/011` kodlarını CUBE/CBRT için ayırıyor ve `111` kodunu genişletme kaçış kodu olarak kullanıyor. Metin ayrıca iki xbitlik işlem-MOD çiftini anlatıyor.

Temmuz'un 5 kodunu Hunter LD, 6 kodunu Hunter ST diye okumuyorum. Temmuz'da ayrılan CUBE/CBRT yerlerini de Hunter'ın önerdiği SHL/SHR'den ayrı tutuyorum. Önceki sayı kelimesi ve bayrak düzeni o sürüme ait, güncel 192 bitlik register'a değil.

CTCPU, Parçalar, Resolver, English

James Augustus, 2026. CC BY 4.0 lisansıyla. https://github.com/JamesAugustus/ctcpu-hunter